Assignment 01

Published | Assign To | Edit

Use of Generative AI Is Not Permitted

Learning Objectives

This assignment aims to develop your skills in technology stack evaluation and cloud-native application development. You will:

Primary Objective: Select an appropriate technology stack for a backend web application (API only) and implement a functional health check API endpoint.

Key Requirements: Your chosen technology stack must align with cloud-native web application principles and demonstrate understanding of modern backend development practices.

Learning Outcomes: Upon completion, you will be able to:

- Evaluate and justify technology stack decisions based on cloud-native requirements
- Implement a basic API endpoint following industry standards
- Apply cloud-native design principles to backend architecture
- Document technical decisions and implementation approaches

Cloud-Native Web Application Requirements

A cloud-native application is designed specifically for cloud computing environments. It leverages distributed architecture and microservices patterns to achieve scalability, resilience, and portability across cloud platforms.

Core Infrastructure Requirements

- **Server Operating System:** Ubuntu 24.04 LTS
  - Provides long-term support and security updates
  - Industry-standard choice for cloud deployments

Development Stack Requirements

- **Programming Language:** Select one high-level language:
  - **Java** (JDK 17+) - Enterprise-grade with robust ecosystem
  - **Python** (3.9+) - Rapid development with extensive libraries
  - **Go** (1.19+) - High performance and cloud-native design

    - **Node.js** (18+) - JavaScript runtime for scalable network applications
    - **PHP** (8.1+) - Web-focused with modern framework support
- **Database Requirements:**
    - **Primary Database:** PostgreSQL 14+ or MySQL 8.0+
    - **Rationale:** Open-source, cloud-compatible, ACID-compliant
    - **Excluded:** MongoDB, MSSQL, Oracle (licensing/compatibility constraints)

Framework Requirements

- **Backend Framework:** Must be open-source and production-ready
    - **Java:** Spring Boot, Quarkus
    - **Python:** FastAPI, Django REST Framework, Flask
    - **Go:** Gin, Echo, Fiber
    - **Node.js:** Express.js, NestJS, Fastify
    - **PHP:** Laravel, Symfony
- **ORM/Data Access Layer:**
    - **Java:** Hibernate, JPA
    - **Python:** SQLAlchemy, Django ORM
    - **Go:** GORM, Ent
    - **Node.js:** Sequelize, TypeORM, Prisma
    - **PHP:** Eloquent (Laravel), Doctrine

Cloud-Native Design Principles

Your implementation should demonstrate:

- **Stateless design** - No server-side session storage
- **External configuration** - Environment-based configuration management
- **Health monitoring** - Comprehensive health check endpoints
- **Graceful degradation** - Proper error handling and fallback mechanisms
- **Observability** - Structured logging and metrics exposure

Excluded Technologies

- **UI/Frontend Components:** Not applicable (API-only backend)
- **Client-side Styling:** Not applicable (no UI layer)

Web Application Development



Bootstrapping Database

The application must implement automated database initialization and schema management to ensure consistent deployments across environments. This approach eliminates manual database setup and supports cloud-native deployment patterns.

Core Requirements

1. **Automatic Schema Management**
   - Application bootstraps the database schema automatically on startup
   - Creates tables, indexes, sequences, constraints, and other database objects
   - Updates existing schema when definitions change (migrations)
   - Handles both initial setup and incremental updates
2. **Zero Manual Intervention**
   - No manual execution of SQL scripts permitted
   - Database setup must be fully automated and reproducible
   - The application should be deployable without a database administrator's intervention
3. **ORM-Driven Approach** *(Highly Recommended)*
   - Leverage ORM frameworks for schema management and migrations
   - Maintain database-agnostic code where possible
   - Use framework-native migration tools

Technology-Specific Implementation Guidelines

Java Applications

- **Hibernate:** Use `hbm2ddl.auto` properties or Flyway/Liquibase integration
- **Spring Boot:** Leverage `spring.jpa.hibernate.ddl-auto` configuration
- **Migration Tools:** Flyway, Liquibase for version-controlled schema changes

Python Applications

- **SQLAlchemy:** Implement `create_all()` for initial setup
- **Alembic:** Use for database migrations and version control
- **Django:** Built-in `migrate` command for automatic schema management

Node.js Applications

- **Sequelize:** Use `sync()` method or Sequelize CLI migrations
- **TypeORM:** Leverage `synchronize` option or migration system
- **Prisma:** Use Prisma Migrate for schema management

Go Applications

- **GORM:** Use `AutoMigrate()` for automatic schema updates
- **Ent:** Leverage built-in schema migration capabilities
- **Custom Solutions:** Database/sql with migration libraries like golang-migrate

PHP Applications

- **Laravel:** Use Eloquent migrations and `php artisan migrate`
- **Symfony/Doctrine:** Implement Doctrine migrations for schema management

RESTful API Requirements

Core API Standards

1. **Content Format:** All API requests and responses must use JSON format exclusively
2. **Interface:** Backend API only - no user interface or frontend components
3. **HTTP Compliance:** All endpoints must return appropriate HTTP status codes per RFC standards

Required Endpoints

Health Check Endpoint

- **Path:** `GET /health` or `/api/health`
- **Purpose:** Application readiness and liveness monitoring
- **Response:** Include service status, database connectivity, and dependencies
- **Status Codes:** `200 OK` (healthy), `503 Service Unavailable` (unhealthy)

API Design Standards

HTTP Methods & Status Codes

- **GET:** `200 OK`, `404 Not Found`, `500 Internal Server Error`
- **POST:** `201 Created`, `400 Bad Request`, `409 Conflict`
- **PUT/PATCH:** `200 OK`, `204 No Content`, `404 Not Found`
- **DELETE:** `204 No Content`, `404 Not Found`

Database Schema Requirements

Health Check Table

Create a dedicated table to track health check requests with the following specifications:

Table Name: `health_checks`

Column Definitions:

Column	Data Type	Constraints	Description
`check_id`	`BIGINT` / `INTEGER`	`PRIMARY KEY`, `AUTO_INCREMENT` / `SERIAL`	Unique identifier for each health check request
`check_datetime`	`TIMESTAMP` / `DATETIME`	`NOT NULL`, `DEFAULT CURRENT_TIMESTAMP`	UTC timestamp when health check was performed

ORM Model Requirements

- **Primary Key:** Auto-generated sequence/auto-increment field
- **Timezone Handling:** Store all timestamps in the UTC timezone
- **Insertion:** Automatically populate `check_datetime` on record creation
- **Indexing:** Consider adding an index on `check_datetime` for query performance

Functional Requirements

1. **Record Creation:** Insert a new record on each health check API call
2. **Data Integrity:** Ensure all timestamps are stored in UTC format
3. **Performance:** Support high-frequency health check requests

Health Check API Requirements

Purpose

Detect when a service instance cannot handle requests and prevent traffic routing to unhealthy instances.

Endpoint: `/healthz`

Method

- **Supported**: GET only
- **Unsupported methods**: Return `405 Method Not Allowed`

Request

- **No payload allowed**: Return `400 Bad Request` if any payload is included
- **Headers**: Standard GET request headers

Response

- **No payload**: Response body must be empty
- **Cache control**: Include `Cache-Control: no-cache` header
- **Status codes**:
  - `200 OK` : Database insert successful
  - `400 Bad Request` : Request contains payload
  - `405 Method Not Allowed` : Non-GET method used
  - `503 Service Unavailable` : Database insert failed

Functionality

1. Insert a record into the health check table
2. Return an appropriate status code based on the insert result
3. Verify database connectivity through the insert operation

Health Check Scope

This implementation focuses on **database connectivity**. Future enhancements may include:

- Downstream API availability
- Resource utilization checks
- Custom business logic validation

Example Requests

Success

```
curl -vvvv http://localhost:8080/healthz
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /healthz HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.1.2
> Accept: */*
>
< HTTP/1.1 200 OK
< Cache-Control: no-cache, no-store, must-revalidate;
< Pragma: no-cache
< X-Content-Type-Options: nosniff
< Date: Wed, 20 Sep 2023 01:18:37 GMT
< Content-Length: 0
<
* Connection #0 to host localhost left intact
```

Failure

```
$ curl -vvvv http://localhost:8080/healthz
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /healthz HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.1.2
> Accept: */*
>
< HTTP/1.1 503 Service Unavailable
< Cache-Control: no-cache, no-store, must-revalidate;
< Pragma: no-cache
< X-Content-Type-Options: nosniff
< Date: Wed, 20 Sep 2023 01:18:57 GMT
< Content-Length: 0
<
* Connection #0 to host localhost left intact
```

405 Method Not Allowed

```
$ curl -vvvv -XPUT http://localhost:8080/healthz
*   Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> PUT /healthz HTTP/1.1
```

```
> Host: localhost:8080
> User-Agent: curl/8.1.2
> Accept: */*
>
< HTTP/1.1 405 Method Not Allowed
< Cache-Control: no-cache, no-store, must-revalidate;
< Pragma: no-cache
< X-Content-Type-Options: nosniff
< Date: Wed, 20 Sep 2023 01:38:11 GMT
< Content-Length: 0
<
*
 Connection #0 to host localhost left intact
```

Assignment Submission Instructions

Submission Steps

1. Create Submission Folder

Create a folder named: `firstname_lastname_neuid_##`

- Replace `##` with the assignment number
- Example: `john_smith_001234567_05`

2. Copy Code into Submission Folder

Copy the source code into this folder. Make sure to include everything required to build and run the code. Do not include downloaded dependencies or binary artifacts you create with your build script.

3. Create and Verify ZIP File

bash

```
# Create zip file
zip -r firstname_lastname_neuid_##.zip firstname_lastname_neuid_##/

# Verify contents by extracting in a different location
mkdir temp_verify
cd temp_verify
unzip ../firstname_lastname_neuid_##.zip
# Check that all repositories and files are present
```

4. Submit to Canvas

- Upload `firstname_lastname_neuid_##.zip` to the correct assignment
- **Resubmissions**: Up to 3 attempts allowed
- Only the latest submission will be graded

Submission Checklist

- Folder named correctly with assignment number
- All organization repositories cloned (not forked repos)
- ZIP file created and verified
- Uploaded to correct the Canvas assignment
- Confirmed the latest submission if resubmitting

Assignment Grading Guidelines

These guidelines are subject to change at the instructor's discretion.

Security & Credentials

Credential Protection

- **Zero Tolerance**: Any credentials, API keys, passwords, or secrets committed to repositories will result in penalties
- **Account Suspension Risk**: Cloud providers may automatically disable accounts when credentials are detected in repositories
- **No Extensions**: Credential-related account issues do not qualify for assignment extensions

Best Practices

- Use environment variables for all sensitive configuration
- Include comprehensive `.gitignore` files
- Use secrets management tools when available

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing

- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures
- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Penalty Structure

Major Violations

- Committed credentials or secrets
- Application crashes during demo

Moderate Violations

- Hard-coded values that cannot be modified
- Persistent 500 errors

Minor Violations

- Style guide violations
- Minor error handling gaps

Web Application

- Students will demo the web application from their laptops. Download the zip uploaded to Canvas and use the submitted code for the demo. Local code cannot be used for the demo.
- APIs can be demoed using any Postman, Restlet, or some other REST client, but not via the browser.
- The application should not have UI.
- Verify the success and failure by shutting down the database server while the application is still running. Restarting the database without restarting the application should work.

- Verify that the only HTTP request method supported is `GET`. Making `POST`, `PUT`, `DELETE`, or `PATCH` requests should return HTTP status code `405 Method Not Allowed`.
- Verify that the request and response do not require a payload. The API response should NOT include a body, and the request should NOT require any query parameters.
- The application connects to either `MySQL` or `PostgreSQL`. No other database is allowed.
- Verify that the code was NOT generated using Generative AI technologies.
- The application should not throw `500 Internal Server` errors.
- The application should not require a restart between API calls.

Demo Expectations

- The application should run smoothly without intervention
- All features must be demonstrable as specified
- Configuration changes should be possible when requested
- Error scenarios should be handled gracefully

Use of Generative AI Is Not Permitted

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Sep 18 at 3pm	Everyone	Sep 11 at 9:30pm	Sep 19 at 3pm

+ Rubric